# Laboratorio 09 Consumo de API con Retrofit


## Leonel Arenas Arias

Desarrollo de aplicaciones móviles · 16 de octubre de 2026

Correo: Leonel.arenas@tecsup.edu.pe


## Objetivo y resultado

Consumir servicios REST con Retrofit y visualizar sus respuestas en una aplicación Jetpack Compose. Se implementó la parte guiada de publicaciones, el ejercicio de recetas y el reto adicional de anfibios. La aplicación presenta los contenidos del catálogo de demostración en español y permite navegar entre las secciones.

## Configuración del proyecto

Se utilizó el proyecto Lab09 con Kotlin 1.9.0, Android Gradle Plugin 8.6.0, SDK de compilación 34 y Android mínimo 24. Se añadió el permiso android.permission.INTERNET en AndroidManifest.xml.

Dependencias añadidas: Retrofit 2.11.0, converter-gson 2.11.0, Navigation Compose 2.7.7, Lifecycle ViewModel Compose 2.6.1 y Coil Compose 2.6.0. Las corrutinas permiten realizar consultas sin bloquear la interfaz.

## Organización MVVM

Vista: MainActivity.kt y ui/moduloPosts.kt muestran las pantallas y reciben acciones. ViewModel: ui/LabViewModel.kt conserva estados observables y ejecuta peticiones en viewModelScope. Datos: data/ApiRepository.kt contiene modelos, interfaces y acceso a los servicios REST.

El usuario inicia una acción que pasa al ViewModel, al repositorio y a Retrofit para consultar la API. La respuesta vuelve al estado del ViewModel y Compose actualiza la pantalla. Se muestran estados de carga, error con reintento y resultados vacíos.

Repositorio: https://github.com/leonel10164/Lab09

# Respuestas de la parte guiada

### 1 Qué otros servicios incluye JSONPlaceholder Mock

Además de publicaciones, este servidor de demostración dispone de recursos como usuarios, comentarios y tareas. Beeceptor también ofrece servidores de ejemplo para empresas, productos y otros escenarios; se debe consultar la ruta y el servidor correspondiente, porque sus catálogos no son idénticos.

### 3.2 Función de @SerializedName

Relaciona una propiedad de Kotlin con la clave de un objeto JSON durante la conversión con Gson. Por ejemplo, @SerializedName("comment_count") permite recibir ese campo en la propiedad commentCount. También evita depender del nombre de la propiedad Kotlin para el contrato de datos.

### 4.2 Función de @GET

Declara que Retrofit debe realizar una petición HTTP GET a la ruta indicada. @GET("posts") solicita el listado; @GET("posts/{id}") incluye un segmento variable que se completa con el argumento marcado con @Path("id").

### 4.3 Función de suspend

Permite que la función se suspenda dentro de una corrutina y reanude su ejecución cuando el resultado esté disponible. En Retrofit, la integración de funciones suspend gestiona la espera de la respuesta sin bloquear el hilo principal. suspend por sí solo no convierte cualquier operación bloqueante en trabajo de segundo plano.

### 5.1.3 Función de rememberNavController

Crea y recuerda un NavHostController en la composición. Este controlador mantiene la navegación y su pila de destinos. Se comparte con el NavHost y con las acciones que llaman a navigate o popBackStack; no se debe crear un controlador nuevo en cada recomposición.

### 5.3.3 Explicación de Contenido

En la guía, Contenido recibe los espacios del Scaffold, el controlador de navegación y el servicio. El Box aplica el padding para evitar que el contenido quede debajo de las barras. El NavHost asocia las rutas inicio, posts y postsVer/{id} con sus pantallas. El argumento id se declara de tipo entero y se entrega al detalle. En esta implementación, esa responsabilidad está integrada en ProgPrincipal9 y las consultas se delegan al ViewModel.

# Estado y despliegue de información

### 6.1.2 Objetivo de LaunchedEffect

Ejecuta una corrutina vinculada a la presencia del composable. Con una clave estable, como Unit, se inicia al entrar en la composición; si cambia una clave, se cancela el bloque anterior y se inicia otro. En el detalle se usa el identificador como clave para solicitar el registro correspondiente. En este proyecto inicia una acción del ViewModel; la petición se ejecuta dentro de viewModelScope.

### 6.1.3 Objetivo de SnapshotStateList

Es una lista observable por Compose. Cuando se leen sus elementos en la composición, las operaciones como añadir o eliminar pueden provocar la actualización de la interfaz. Se crea con mutableStateListOf. En la solución se utiliza un estado observable UiState que contiene listas inmutables, de modo que el ViewModel publica un resultado completo.

### 6.1.3 Para qué sirve LazyColumn

Muestra una lista vertical desplazable componiendo principalmente los elementos necesarios para el área visible. Evita crear de una vez toda una lista extensa. En publicaciones y recetas se emplean identificadores como claves para mantener la identidad de cada elemento.

### 6.1.4 Cómo se muestra la publicación seleccionada

Al pulsar una tarjeta se navega a postsVer/{id}. Navigation obtiene el identificador entero y LaunchedEffect(id) solicita el detalle al ViewModel. El repositorio ejecuta GET posts/{id}; Gson convierte el JSON en PostModel. Al llegar la respuesta se publica el estado y la pantalla muestra título, autor, cuerpo y los campos opcionales disponibles.

### 6.1.5 Aplicación de MVVM

Sí. Las pantallas se ocupan de presentar y notificar acciones; LabViewModel administra el estado de carga, resultado y error; ApiRepository encapsula el acceso a Retrofit. Esta separación evita que la UI configure clientes de red durante cada recomposición y facilita mantener o sustituir los servicios.

# Adaptación al español

Las API devuelven texto en inglés. ContenidoEspanol.kt adapta la presentación de los diez posts, cinco recetas y seis anfibios del catálogo utilizado. Se conservan identificadores, imágenes y valores numéricos recibidos. El catálogo de recetas se solicita con limit=5; no representa la totalidad del recurso. Las traducciones son locales y no requieren una API de traducción.

# Evidencias de publicaciones

Se consultó el listado de publicaciones y se abrió el registro 1 mediante su identificador. Las capturas muestran los datos cargados desde el servicio y su presentación en español.




Figura 1 Lista de publicaciones de la API




Figura 2 Detalle de la publicación seleccionada

Ruta consultada: https://json-placeholder.mock.beeceptor.com/posts y /posts/1. El detalle conserva el autor, la cantidad de comentarios y el enlace proporcionado por el servicio.

# Ejercicio 1 Recetas

Se eligió el recurso recipes de DummyJSON. La pantalla permite buscar por nombre o tipo de cocina y abrir una receta. Se muestran fotografía, dificultad, valoración, tiempo total, porciones, ingredientes y pasos de preparación.

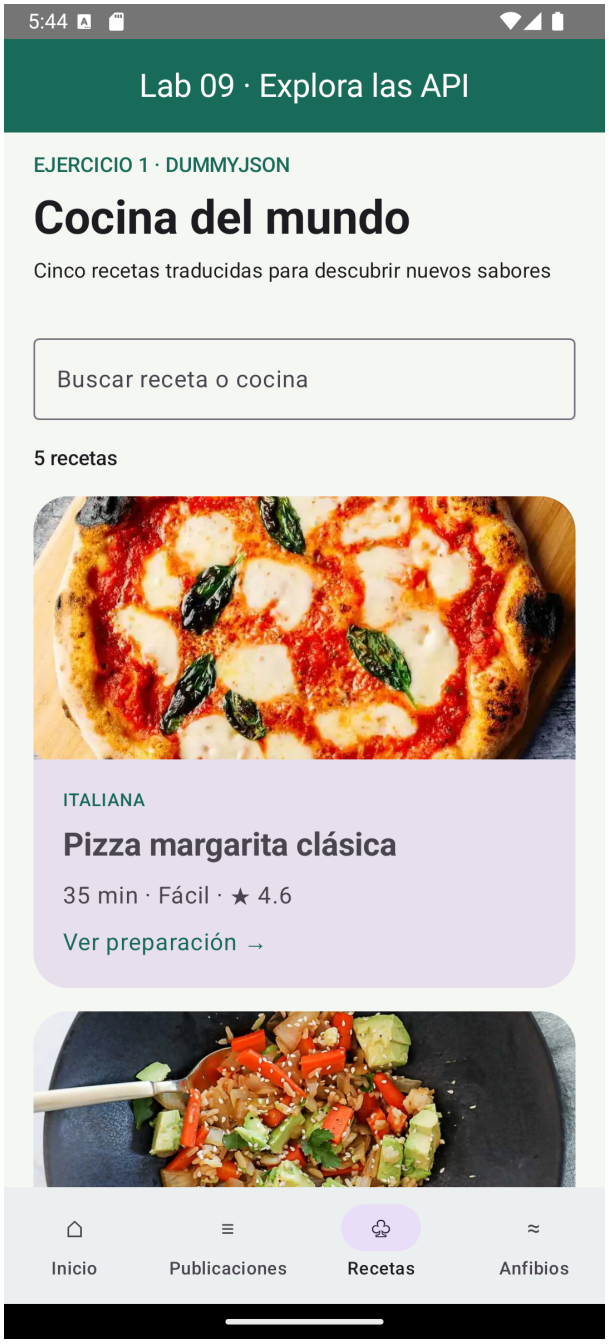



Figura 3 Catálogo de cinco recetas en español

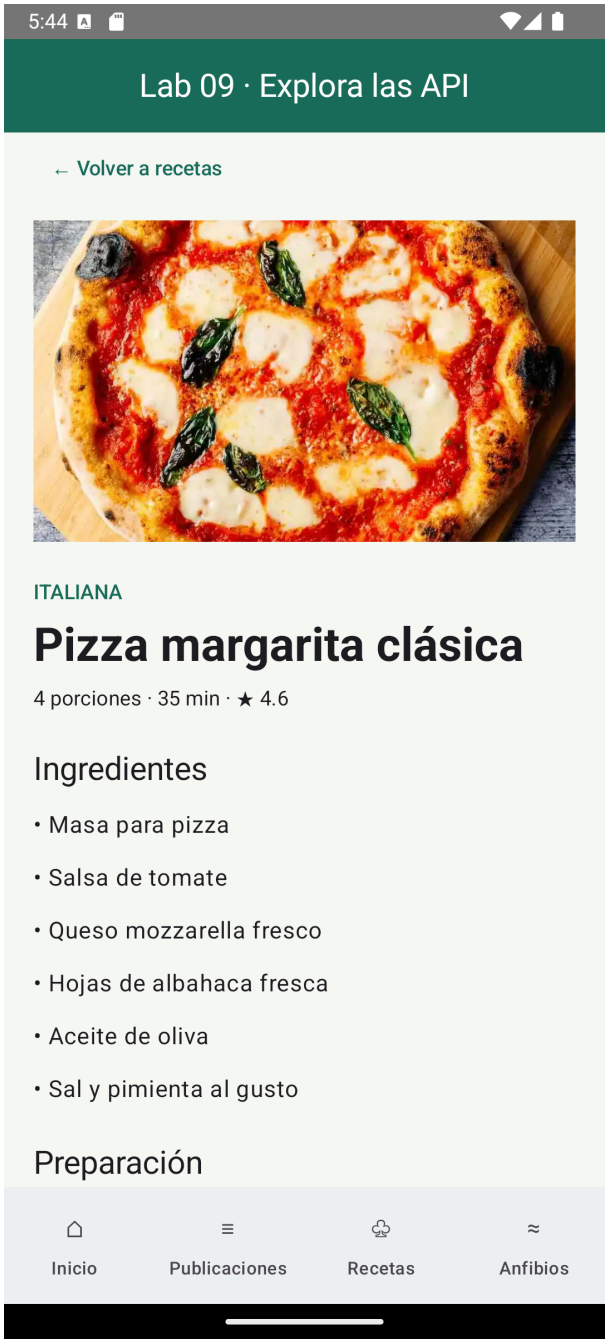



Figura 4 Detalle de la receta seleccionada

El listado usa GET recipes?limit=5 y el detalle usa GET recipes/{id}. Coil carga las imágenes devueltas por la API. La búsqueda filtra las cinco recetas descargadas.

## Preparación y reto de anfibios

El detalle de recetas admite desplazamiento para leer toda la preparación. Como reto adicional se implementó un catálogo de anfibios con fotografía, nombre, tipo y descripción.

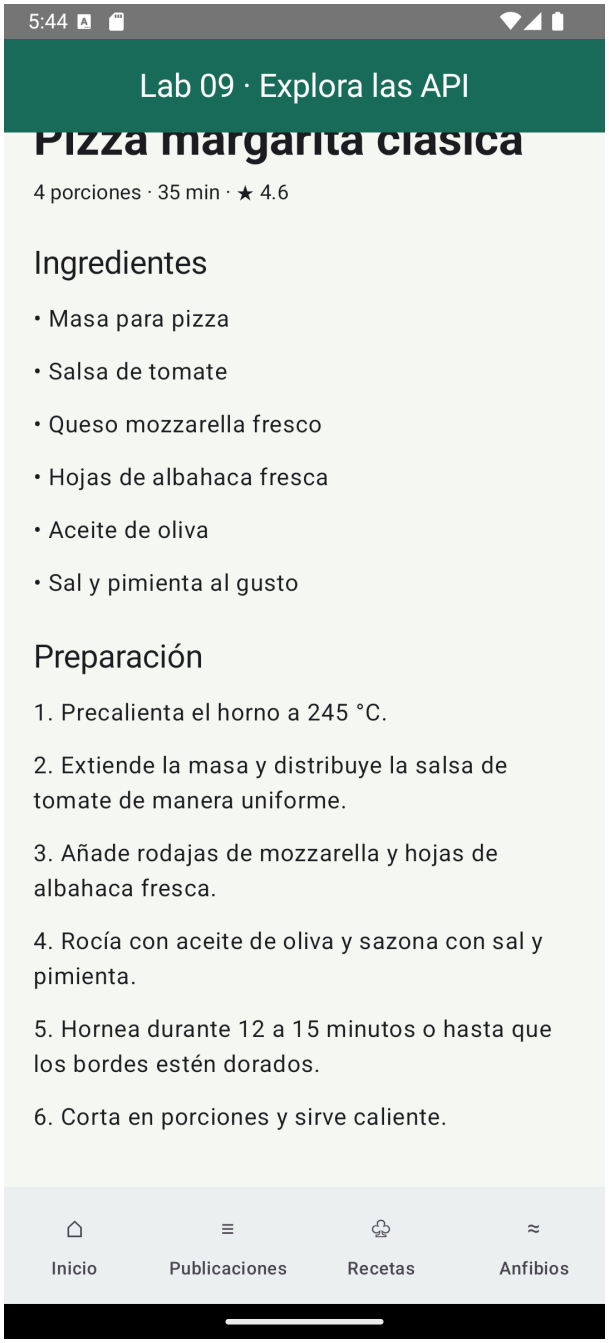



Figura 5 Ingredientes y preparación en español




Figura 6 Catálogo de anfibios con imágenes

El reto utiliza GET https://android-kotlin-fun-mars-server.appspot.com/amphibians. Su JSON se transforma en una lista de Amphibian y se presenta mediante LazyColumn.

# Ejecución y verificación

La aplicación se compiló mediante assembleDebug y se ejecutó en el emulador Medium Phone con Android API 35. También se ejecutó testDebugUnitTest; la prueba unitaria inicial del proyecto pasó. La validación funcional se realizó navegando por las pantallas del emulador y observando las respuestas de las API.

Se verificaron la lista de publicaciones, la consulta por identificador, el catálogo de recetas, su detalle con ingredientes y preparación y el catálogo de anfibios. Las capturas proceden de esa ejecución. El video laboratorio-es.mp4 documenta el recorrido por las secciones.

# Pregunta de observación

La parte más desafiante del desarrollo fue coordinar las peticiones de red con la navegación y el estado de Compose. Una consulta lanzada directamente al dibujar una pantalla puede repetirse cuando se recompone; además, el resultado tarda y puede fallar. Se resolvió separando el acceso a la API en un repositorio y centralizando el estado en un ViewModel. La pantalla inicia la carga con LaunchedEffect y presenta carga, datos o error con opción de reintento.

# Uso de inteligencia artificial

Se utilizó IA para interpretar la guía, proponer la estructura MVVM, generar y ajustar código Kotlin, traducir el contenido del catálogo y preparar el informe. También se utilizó para ejecutar la compilación y automatizar la navegación del emulador, las capturas y la grabación. Las evidencias muestran la aplicación ejecutándose; el código y las explicaciones deben revisarse para comprender su funcionamiento.

# Fuentes de consulta

**Beeceptor y servicios de ejemplo**

Consultar documentación oficial

**Documentación de recetas de DummyJSON**

Consultar documentación oficial

**Efectos en Jetpack Compose**

Consultar documentación oficial

**Reto de anfibios de Android**

Consultar documentación oficial